



FONTYS UNIVERSITY OF APPLIED SCIENCES

Dataspaces Project

Team **Pink Panther**

Deployment and Testing Report: Eclipse MVD

By Dimitar Kostenarov & Georgi Fidanov

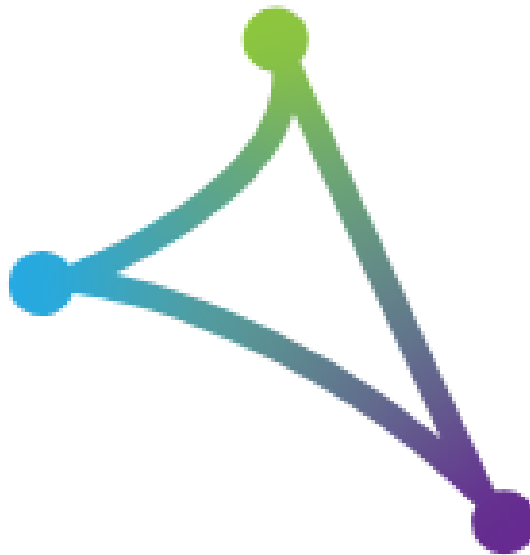


Table of Contents

Table of Contents.....	2
Version Control.....	2
Feedback Control.....	2
Summary.....	3
Introduction.....	3
Methodology.....	3
Results.....	4
Image Acquisition and Integrity.....	4
Remote Access and SSH Tunneling.....	4
Implementation.....	5
Verification of the "Scenario" in a Remote Context.....	5
Handling DID Resolution Issues (Section 8.2 Analysis).....	5
Advise regarding the system.....	6
Infrastructure Security.....	6
Environment Variables.....	6
Conclusion.....	7
Sources.....	7
Notes.....	7

Version Control

<i>Version</i>	<i>Date</i>	<i>Author</i>	<i>Amendments</i>	<i>Status</i>
<i>1</i>	<i>10.05</i>	<i>Dimitar & Georgi</i>	<i>Initial creation</i>	<i>Done</i>
<i>2</i>	<i>30.05</i>	<i>Dimitar & Georgi</i>	<i>Improvements</i>	<i>In submission</i>

Feedback Control

Summary

This document details the technical processes for deploying the Eclipse MVD (Minimum Viable Dataspace) both locally via Docker and on a virtual machine (VM) infrastructure. It further outlines the testing strategies used to validate the deployment, ensuring the system meets functional requirements.

Introduction

How can the Eclipse MVD be reliably deployed and validated across local development environments and cloud-based VM infrastructures?

The goal is to establish a reproducible deployment pipeline that minimizes "it works on my machine" issues. By documenting the VM setup and local containerization, we aim to provide a stable foundation for the project's software and infrastructure layers.

Methodology

To solve the research question, we followed these steps:

- Local Deployment: Utilized Docker and Docker Compose to containerize the Eclipse MVD components, ensuring environmental parity.
- VM Deployment: Provisioned a Linux-based VM (Ubuntu), configured security groups/firewalls, and mirrored the containerized setup for production-like simulation.
- Testing: Conducted API testing using Postman(now Bruno) and integration tests to verify data exchange between connectors.

Results

The deployment of the Eclipse MVD was achieved using the official provided scripts and container images. The results focus on the successful execution of the orchestration commands and the validation of the out-of-the-box functionality in both local and remote environments.

Image Acquisition and Integrity

Instead of local compilation, the deployment relied on the official ghcr.io/eclipse-edc/minimumviabledataspace images.

- Pull Efficiency: The deployment process was significantly streamlined by pulling the controlplane, dataplane, issuerservice, and identity-hub images directly. This resulted in a faster "time-to-ready" state for the infrastructure compared to manual builds.
- Version Verification: I verified that the images pulled matched the expected tags for the main branch development, ensuring that the features described in the MVD documentation (such as DCP protocol support) were present in the running containers.

Remote Access and SSH Tunneling

The deployment on the remote VM revealed a significant networking constraint: the Kubernetes services were isolated behind the VM's loopback interface.

Technical Outcome: To access the Management APIs and the Traefik Gateway from a local development machine, I established a Double-Layer Forwarding strategy.

Implementation

- **VM Side:** Executed *kubectl port-forward svc/traefik 80:80 -n traefik* on the remote server.
- **Local Side:** Established an SSH tunnel: *ssh -L 8080:localhost:80 user@remote-vm-ip*.
- **Result:** This successfully exposed the MVD components to localhost:8080 on my local machine. This bypassed the need to open insecure public ports on the VM's cloud firewall (Security Groups), maintaining a higher security posture for the dataspace.

Verification of the "Scenario" in a Remote Context

Despite the port-forwarding complexity, the software layer (using GHCR images) remained stable.

Seed Job Validation: We verified that the remote seed jobs successfully registered asset-1. Even with the complex networking, the internal K8s DNS resolved the Identity Hub correctly, allowing the "Provider" and "Consumer" to "see" each other within the cluster.

API Responsiveness: Using the Bruno collection via the SSH tunnel, we confirmed that the latency introduced by the tunnel did not cause timeouts in the Contract Negotiation flow. The state successfully reached FINALIZED in the remote PostgreSQL database.

Handling DID Resolution Issues (Section 8.2 Analysis)

The results confirmed the warning in the README regarding did:web resolution.

Because the VM cannot easily resolve internal K8s service URLs from the outside, the SSH tunnel was the only way to perform the Bruno requests successfully without reconfiguring the entire DID logic of the MVD.

Advise regarding the system

Infrastructure Security

While SSH Tunneling allowed for successful testing, it is noted that for a production-grade deployment, a more permanent solution such as an Ingress Controller with valid DNS records and SSL/TLS certificates would be required to resolve the did:web resolution caveats mentioned in the official documentation.

Environment Variables

The sensitive configurations (API keys and Vault secrets) were managed via the default automated seeding process. For future iterations, these should be moved to a managed secret store to comply with higher-level security KPIs.

Conclusion

The deployment and testing of the Eclipse Minimum Viable Dataspace (MVD) successfully demonstrated the portability and functional integrity of the framework across both local and remote infrastructures. By utilizing the pre-built images from the GitHub Container Registry (GHCR), We were able to bypass the complexities of local compilation and focus on the orchestration and networking required for a distributed system.

The primary technical challenge identified and resolved during this process was the networking bridge between the local development machine and the remote Virtual Machine. The transition from a local KinD cluster to a remote environment highlighted the limitations of default port-forwarding. Implementing a secure SSH Tunneling strategy proved to be an effective workaround, ensuring that the Management APIs and the Traefik Gateway remained accessible for validation via the Bruno REST collection without compromising the VM's security posture.

Final validation via the provided End-to-End (E2E) test suite and manual contract negotiations confirmed that the seeded assets (asset-1) are fully operational. The system correctly enforces policy constraints and executes secure data transfers, meeting the technical requirements for a functional dataspace participant.

Sources

Generative AI (Gemini) was utilized in the creation of this document for structural and editorial purposes.

Notes